

Mission 1 of 4 — Project Discovery

Suggested time: ~15 minutes | Deliverable: Technology selection & requirement analysis

You are the Graduate Software Engineer assigned to Project Phoenix: rebuilding the technology platform for NimbusCart, a growing online marketplace connecting local sellers with regional buyers.

Client Requirements

- Sellers can register, create a storefront, and list products with images, prices, and descriptions.
- Buyers can browse, search, and filter products, then complete a purchase through checkout.
- Every user (seller or buyer) logs in securely to a single account.
- Buyers and sellers receive notifications for key order events (order placed, shipped, delivered).
- Checkout must support real payment processing, not just a placeholder form.
- The operations team needs basic visibility into whether the platform is healthy at any given time.

Business Constraints

- Budget cap: 9,500 budget units per month total (≈ ■8,00,000/month), across every technology tier combined — this cap applies for the rest of the simulation too.
- Team size: a small engineering team with limited dedicated DevOps capacity — prefer managed services over heavy self-managed infrastructure unless there's a strong reason not to.
- Launch scope: initial launch is single-region, with moderate expected traffic — this is a pilot, not day-one hyperscale.
- Compliance: any component touching payment card data must be PCI-DSS compliant.
- Timeline: the client wants a working platform soon; favor proven, fast-to-integrate options over long build-outs.

Technology Inventory

Review the available options below. Unfamiliar terms are covered in the glossary at the end of this page.

Category	Req.	Option	Cost	Engineering Note
Frontend App Tier	Required	React / Next.js SPA (CDN Hosted)	150	Fast static delivery via global edge CDN with serverless SSR support.
		Server-Rendered Templates (Monolithic UI)	300	Couples UI closely to backend app instances; harder to scale independently.
Backend API Engine	Required	Auto-Scaling Managed Compute (VM / Containers)	1500	Managed environment with low ops overhead for application services.
		Managed Kubernetes Cluster	3000	High operational complexity and cost; steep learning curve for small teams.

Category	Req.	Option	Cost	Engineering Note
		Serverless Functions (FaaS)	600	Cost-effective for sporadic traffic, but vulnerable to cold starts.
Primary Database	Required	Managed Relational DB — Medium + Read Replica	900	Separates primary read/write traffic while maintaining transaction integrity.
		Managed Relational DB — Multi-AZ + 2 Replicas	2600	High availability setup, best suited for heavy production loads.
		Managed NoSQL Document Store	800	Great for flexible schemas; requires extra care for relational data integrity.
Identity & Authentication	Required	Managed Auth Service (OAuth2 / OIDC)	120	Offloads session security, MFA, and credential storage compliance.
		Custom JWT Auth Service	50	Requires full team maintenance for security patches and token rotation.
Notifications & Message Queues	Required	Managed SQS / Message Broker	100	Asynchronously decouples background tasks (emails, webhooks, processing).
		Distributed Event Stream (Kafka-class)	1400	High throughput event stream; high maintenance overhead.
Observability & Analytics	Optional	Managed Log Aggregation & Metrics Suite	250	Provides real-time dashboards, alerting, and APM tracing.
		Basic Cloud Provider Metrics	50	Includes fundamental CPU/Memory metrics without deep application tracing.
Payment Gateway Integration	Optional	PCI-DSS Certified Payment API	0	Per-transaction fee structure — zero infrastructure management overhead.
		In-House Vault Storage	500	Violates compliance guidelines unless rigorous audit standards are met.

The AI Recommendation Engine, Caching, Storage & CDN, and Disaster Recovery tiers are not required for this mission — they appear later in the Technology Inventory as the project evolves.

Deliverable

- Select one option for each required category above.
- For each choice, write a brief justification (1–2 sentences) tying it back to the client requirements or business constraints.
- Write a short requirement analysis summary (2–4 sentences) explaining how your selections meet the client's needs within the budget cap.

Evaluation Focus: Requirement Analysis • Technology Selection

■ Hints — Mission 1 – Project Discovery

- Separate the client's functional requirements (what the platform must do) from the business constraints (budget, team size, compliance, timeline) — you'll need both to justify a choice.
- For every required category, ask: does this option fit the budget cap, the team's ability to operate it, and the compliance needs — not just "which one sounds more powerful"?

- A smaller, simpler option is a legitimate choice for a small team on a tight timeline. Bigger isn't automatically better here.

■ Hints — General

- There is no single correct architecture. The jury is evaluating your reasoning and clarity, not whether you matched a hidden answer key.
- Re-read the client requirements and business constraints before opening the Technology Inventory — it's easy to pick a popular-sounding option that doesn't actually fit the brief.
- The 60 minutes are split across four missions. Pace yourself; don't spend so long perfecting Mission 1 that later missions get rushed.
- Every deliverable asks for a short written justification somewhere — keep a running note of *why* you chose each component, in plain language.
- Cost and complexity are trade-offs, not penalties. A more expensive or more complex option can be the right call if the requirements justify it — say so.

■ Glossary — terms used on this page

Term	What it means
SPA (Single Page App)	A frontend built as one page that updates its content dynamically, instead of reloading a new page for every screen.
CDN (Content Delivery Network)	A network of servers spread across locations that cache and serve content closer to the user, reducing load time.
SSR (Server-Side Rendering)	Generating a webpage's HTML on the server before sending it to the browser, rather than building it in the browser.
Monolithic Architecture	A design where the UI, business logic, and other concerns run together as one deployable unit, rather than as separate services.
Auto-Scaling Compute	Backend compute capacity (VMs or containers) that automatically increases or decreases based on traffic load.
Kubernetes	A system for running and managing many containers across machines, handling scheduling, scaling, and recovery automatically.
Serverless Functions (FaaS)	Small backend functions that run only when triggered and scale automatically, without managing servers directly.
Cold Start	The delay before a serverless function responds when it has to start up from an idle state.
Relational Database (RDBMS)	A structured database organized into tables with defined relationships, typically queried using SQL.
Read Replica	A copy of a database used to handle read queries, so the primary database is freed up mainly for writes.
Multi-AZ (Multi Availability Zone)	Running infrastructure across more than one physically separate data center location, so a failure in one doesn't take the system down.

NoSQL Document Store	A database that stores data as flexible, schema-less documents rather than fixed rows and columns.
OAuth2 / OIDC	Standard protocols that let an app confirm a user's identity and permissions, often via a third-party identity provider, without the app storing passwords itself.
JWT (JSON Web Token)	A compact, signed token used to represent a user's identity and permissions between requests.
MFA (Multi-Factor Authentication)	Requiring more than one form of proof of identity to log in, such as a password plus a one-time code.
Message Queue	A component that holds messages between systems so tasks can be processed asynchronously instead of immediately, blocking the user.
Kafka-class Event Stream	A high-throughput system for publishing and consuming continuous streams of events, used when many services need to react to the same events in real time.
PCI-DSS	A security standard that any system handling card payment data must follow to stay compliant.
APM (Application Performance Monitoring)	Tooling that tracks how an application is performing in production — response times, errors, and traces through the system.